Analyzing College Basketball Matchups through Team Archetypes

Seth Jernigan, Ian Stewart, Daniel Lott, Michael Masterton, Michael Borg

2025-05-08

Introduction

College basketball is a strategic game where a team's success depends on more than just athletic talent: team strategy and composition are important as well. In college basketball, two teams of five players at a time compete by trying to outscore the other team. To score, they must shoot a basketball through the opposing team's hoop, worth 1-3 points for each basket depending on where the shot is from (1 point for free throws after a foul, 2 for field goals within the 3 point line, and 3 points for field goals behind the three point line). The college basketball games are played in two 20-minute halves. In this project we're looking specifically at the Power 5, which has 5 conferences (ACC, Big Ten, Big 12, Pac-12, and SEC), and 70 total teams. We'll be looking at the seasons from 2019-2024.

One of the most interesting parts of college basketball is the difference in playing styles between teams. Some teams have offenses with fast pace and consistent 3-point shooting as their focus, while other teams might have a more center dominant offense, and there's also differences in defensive styles. We're aiming to understand how these styles (which we'll refer to as archetypes) perform against each other, which will offer insights into matchup dynamics and competitive advantages in certain matchups.

We will use K-Means clustering to group the Power 5 teams into archetypes based on advanced performance metrics such as offensive and defensive efficiency, effective field goal percentage, pace, etc. After identifying these archetypes, we'll analyze their performance in matchups to explore whether certain styles consistently outperform others.

Data

The data for our college basketball matchup analysis combines multiple datasets focusing on Power 5 conference teams across 6 seasons (2019-2024). We utilize advanced basketball metrics from KenPom.com, which provides detailed team statistics that go beyond traditional box score numbers. We also incorporate game results data from ncaahoopR, an R package that provides access to NCAA basketball game results data.

Data Sources

Our primary data comes from several related datasets from KenPom.com:

- Team Advanced Metrics: We collected comprehensive team statistics including the "Four Factors" (effective field goal percentage, turnover rate, offensive rebounding percentage, and free throw rate) for both offense and defense.

- Efficiency and Tempo: Each team's adjusted offensive and defensive efficiency ratings, along with pace metrics that describe how quickly teams play.

- Player and Team Attributes: Height data, experience levels, and bench utilization statistics.

- Game Results: Outcomes of individual games, including scores, location (home/away/neutral), and dates.

- Point Distribution Data: Breakdowns of how teams score their points (free throws, 2-pointers, 3-pointers).

In addition, we have compiled game results data from the ncaahoopR repository from regular season games within each of these seasons. These game results allow us to analyze performance between teams of different 'archetypes' throughout this timeframe.

Data Processing

Data preprocessing played a significant role in allowing the analysis of college basketball teams across the timeframe of interest. The first step in this preprocessing was joining each KenPom datafile in a way that produced a individual row for every team season pair, with all of their corresponding advanced metrics associated with that row. We merged 6 seasons of data (2019-2024) for each metric category into these combined datasets. We then filtered the data to focus only on teams in the Power 5 conferences (SEC, Pac-12, ACC, Big Ten, and Big 12). Next, we renamed columns to create consistent naming conventions that distinguish between offensive and defensive metrics. Lastly, the separate metric categories were joined together using team names and seasons as matching keys. The extent of this process can be viewed within data_analysis.Rmd, with further cleaning below, but was not included in this paper for conciseness.

Further, we scraped the ncaahoopR repository to create a dataset of all Power 5 game results from within this timeframe. This process was quite simple yet computationally extensive, and is detailed within the ncaahoopR_sample.Rmd file.

Finally, in order to prepare our data to be used to create team 'archetypes', we finalized cleaning with renaming and cropping features within the dataset, which can be seen below.

```
library(tidyverse)
```

Read in Data

```
## Warning: package 'tidyverse' was built under R version 4.3.3

## Warning: package 'ggplot2' was built under R version 4.3.3

## Warning: package 'tidyr' was built under R version 4.3.3

## Warning: package 'readr' was built under R version 4.3.3

## Warning: package 'dplyr' was built under R version 4.3.3

## Warning: package 'stringr' was built under R version 4.3.3

## Warning: package 'lubridate' was built under R version 4.3.3

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(ggplot2)
library(gridExtra)
```

```
## Warning: package 'gridExtra' was built under R version 4.3.3
```

```
##
## Attaching package: 'gridExtra'
##
## The following object is masked from 'package:dplyr':
##
##     combine
```

```r
library(fmsb)
```

```
## Warning: package 'fmsb' was built under R version 4.3.3
```

```r
library(tibble)
library(purrr)
library(reshape2)
```

```
## Warning: package 'reshape2' was built under R version 4.3.3
```

```
##
## Attaching package: 'reshape2'
##
## The following object is masked from 'package:tidyr':
##
##     smiths
```

```r
mergeDataStr <- paste0(getwd(), "/data/merged_data.csv")
mergeData <- read.csv(mergeDataStr)
```

```r
p5Conf <- c("SEC", "P12", "ACC", "B10", "B12")

p5Data <- mergeData %>%
  filter(Conf %in% p5Conf) %>%
  rename(
    # Four Factors
    off_eFGPct = eFGPct.y,
    def_eFGPct = eFGPct.x,
    off_eFGPct_Rank = RankeFGPct.y,
    def_eFGPct_Rank = RankeFGPct.x,
    off_TOPct = TOPct.y,
    def_TOPct = TOPct.x,
    off_TOPct_Rank = RankTOPct.y,
    def_TOPct_Rank = RankTOPct.x,
    off_ORPct = ORPct.y,
    def_ORPct = ORPct.x,
    off_ORPct_Rank = RankORPct.y,
    def_ORPct_Rank = RankORPct.x,
    off_FTRate = FTRate.y,
    def_FTRate = FTRate.x,
    off_FTRate_Rank = RankFTRate.y,
    def_FTRate_Rank = RankFTRate.x,
```

```
    # Misc [Offense/Defense]
    off_FG2Pct = FG2Pct.x,
    def_FG2Pct = OppFG2Pct.x,
    off_FG2Pct_Rank = RankFG2Pct.x,
    def_FG2Pct_Rank = RankOppFG2Pct.x,
    off_FG3Pct = FG3Pct.x,
    def_FG3Pct = OppFG3Pct.x,
    off_FG3Pct_Rank = RankFG3Pct.x,
    def_FG3Pct_Rank = RankOppFG3Pct.x,
    off_FTPct = FTPct.x,
    def_FTPct = OppFTPct.x,
    off_FTPct_Rank = RankFTPct.x,
    def_FTPct_Rank = RankOppFTPct.x,
    off_BlockPct = BlockPct.x,
    def_BlockPct = OppBlockPct.x,
    off_BlockPct_Rank = RankBlockPct.x,
    def_BlockPct_Rank = RankOppBlockPct.x,
    off_FG3Rate = FG3Rate.x,
    def_FG3Rate = OppFG3Rate.x,
    off_FG3Rate_Rank = RankFG3Rate.x,
    def_FG3Rate_Rank = RankOppFG3Rate.x,
    off_ARate = ARate.x,
    def_ARate = OppARate.x,
    off_ARate_Rank = RankARate.x,
    def_ARate_Rank = RankOppARate.x,
    off_StlRate = StlRate.x,
    def_StlRate = OppStlRate.x,
    off_StlRate_Rank = RankStlRate.x,
    def_StlRate_Rank = RankOppStlRate.x,
    off_NSTRate = NSTRate.x,
    def_NSTRate = OppNSTRate.x,
    off_NSTRate_Rank = RankNSTRate.x,
    def_NSTRate_Rank = RankOppNSTRate.x
  ) %>%
  select(-ends_with(".y"))
```

Filter Data to Power 5, Rename Columns

Feature Selection

For our clustering analysis, we focused on metrics that describe a team's playing style. We removed ranking variables and other columns not directly relevant to our analysis:

```
pca_data <- p5Data %>%
  select(-Conf, -seed) %>%
  select(-ends_with("Rank")) %>%
  select(-starts_with("Rank")) %>%
  select(-DFP.x) %>%
  select(-ends_with("rk"))
```

We further refined our datasets for offensive and defensive clustering, as shown in methods section below.

The final dataset includes 400+ team-seasons, 45 performance metrics per team, and over 11,000 games with Power 5 teams. This dataset provides the foundation for our archetype identification and matchup analysis, allowing us to identify patterns in team playing styles and evaluate how different stylistic approaches perform

against one another.

Methods and Analysis

To group college basketball teams on similar characteristics, we applied K-Means clustering to advanced team metrics obtained from KenPom.com. The decision on how to create these groupings, and what metrics to use to define them, was one that had a variety of options. Ultimately, we chose to perform separate offensive and defensive clusterings. This decision was driven by our focus on quantifying favorable and unfavorable matchups, as well as the limitations existing in both the dataset and modeling methodology.

A basketball game between Team 1 and Team 2 involves two distinct matchups: Team 1's offense versus Team 2's defense, and Team 1's defense versus Team 2's offense. Beyond these two matchups, complex interactions between them exist, including pace of play and shot selection, which can meaningly influence the outcome of the game. However, quantifying these interactions is difficult.

First, data limitations constrain our ability to evaluate enough game outcomes to produce statistically significant insights. Specifically, each team in a game has particular offensive and defensive styles or 'archetypes', and every matchup reflects a unique interaction between these two sets of archetypes. Because each archetype set pairing occurs relatively infrequently, there isn't enough data to analyze each one, and the interaction encapsulated by it, individually.

Secondly, while an alternate approach is evaluating each offensive versus defensive matchup within a game sepearately and then modeling their interaction, doing so involves significant complexities, modeling methodologies, and interpretation detriments, that did not align with the scope or focus of this project.

As a result of these considerations, we opted to individually cluster teams into offensive and defensive clusters based on their offensive and defensive related metrics. This decision allowed us to reduce the number of features within each clustering iteration, reducing the effect of the curse of dimensionality, and improving performance of the K-Means clustering algorithm. The trade-off, though, is that instead of producing a single metric to evaluate a game matchup, we obtain two separate metrics that quantify the matchup between each of the two offensive versus defensive matchups within a game.

First, we assign each team in the dataset to both an offensive and defensive cluster. Initially, this involves determining an optimal amount of clusters, K, since K-Means clustering cannot infer this from unlabeled data. To tune this parameter, we evaluated a range of K values and measured their within cluster sum of squared errors at each step, which essentially quantifies how far away the data points assigned to a specific cluster are from the cluster's center. Minimizing this error improves cluster compactness, ensuring more and more certainly that the data points are assigned to the correct cluster center. In practice, though, the optimal K is unable to be found, as increasing K always results in a decrease in within cluster sum of squared errors.

To address this, we employed the elbow method, which involves plotting within cluster sum of squared errors for the range of values of K tested. The idea of this method is that if you can identify an 'elbow', a point in which the amount of decrease in error significantly decreases, the K value corresponding to this value is a good choice for K. The 'elbow' indicates a point where within cluster sum of squared errors is close to minimized, but doesn't allow choosing of too high a K value to overfit the data with. Below, we create elbow plots for both offensive and defensive clustering.

Offense

```
# Minimized Offensive Data Frame
offensiveDataMin <- pca_data %>%
  select(
    # Team Identification
    TeamName,
    Season,
```

```
    # Eff_Tempo
    tempo_adj,
    pos_len_off,
    off_eff_adj,

    # misc_offense
    off_FG2Pct,
    off_FG3Pct,
    off_FTPct,
    off_BlockPct,
    off_FG3Rate,
    off_ARate,
    off_StlRate,
    off_NSTRate,

    # height

    HgtEff,
    Exp,
    Bench,
    Pts1,
    Pts2,
    Pts3,
    Pts4,
    Pts5,

    # points dist
    Off_1,
    Off_2,
    Off_3,

    # four factors
    off_eFGPct,
    off_TOPct,
    off_ORPct,
    off_FTRate
  )
```

Create Minimized Offensive Dataset

Run K Means Clustering varying k to select using Elbow Method

```
# Run K Means clustering for k = 1:n to produce elbow plot for number of cluster selection

set.seed(123)

n <- 25

offensiveDataCluster <- offensiveDataMin %>% select(-TeamName, -Season)
off_scaled_cluster <- scale(offensiveDataCluster) %>% as.data.frame()

off_scaled <- bind_cols(
  offensiveDataMin %>% select(TeamName, Season),
  off_scaled_cluster
```

```
)

wss <- numeric(n)

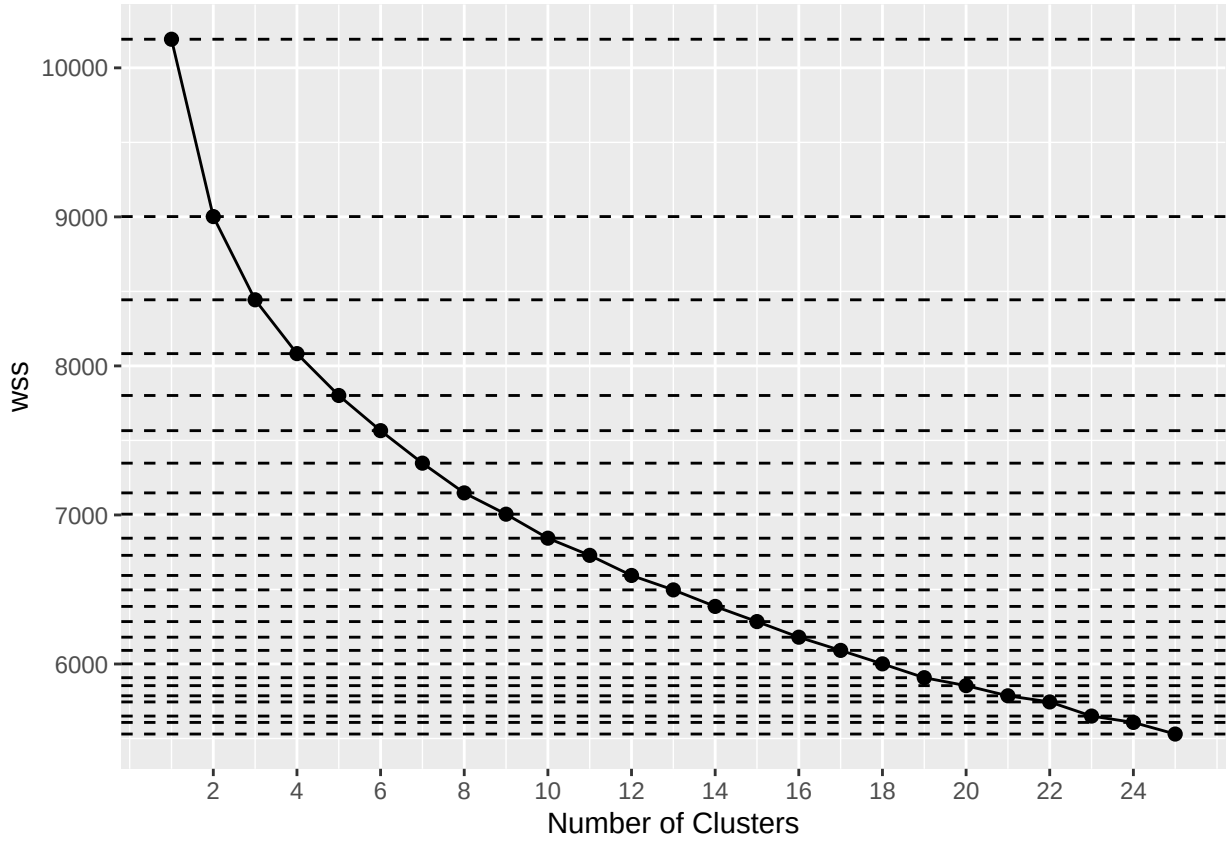
for (k in 1:n){
  km <- kmeans(off_scaled_cluster, centers = k, nstart = 20)

  wss[k] <- km$tot.withinss
}

wss_df <- tibble(clusters = 1:n, wss = wss)

elbow_plot <- ggplot(wss_df, aes(x = clusters, y = wss, group = 1))+
  geom_point(size = 2)+
  geom_line()+
  scale_x_continuous(breaks = seq(2, n, 2)) +
  xlab('Number of Clusters')+
  geom_hline(
    yintercept = wss,
    linetype = 'dashed'
  )

elbow_plot
```



Defense

```r
defensiveDataMin <- pca_data %>%
  select(
    # Team Identification
    TeamName,
    Season,

    # Eff_Tempo
    tempo_adj,
    pos_len_def,
    def_eff_adj,

    # misc_defense
    def_FG2Pct,
    def_FG3Pct,
    def_FTPct,
    def_BlockPct,
    def_FG3Rate,
    def_ARate,
    def_StlRate,
    def_NSTRate,

    # height
    HgtEff,
    Exp,
    Bench,

    # points dist
    Def_1,
    Def_2,
    Def_3,

    # four factors
    def_eFGPct,
    def_TOPct,
    def_ORPct,
    def_FTRate
  )
```

Create Minimized Defensive Dataset

Run K Means Clustering varying k to select using Elbow Method

```r
# Run K Means clustering for k = 1:n to produce elbow plot for number of cluster selection

set.seed(123)

n <- 25

defensiveDataCluster <- defensiveDataMin %>% select(-TeamName, -Season)
def_scaled_cluster <- scale(defensiveDataCluster) %>% as.data.frame()

def_scaled <- bind_cols(
  defensiveDataMin %>% select(TeamName, Season),
```

```
  def_scaled_cluster
)

wss <- numeric(n)

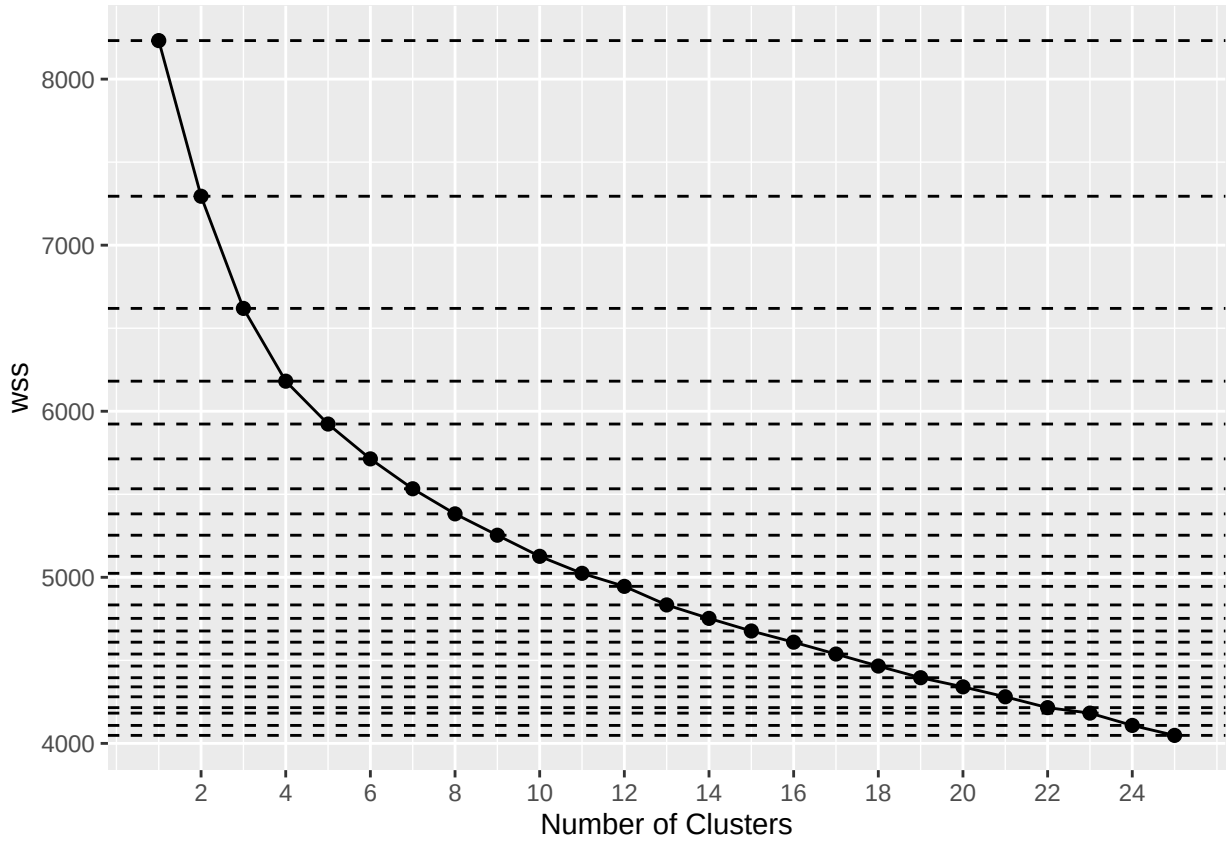
for (k in 1:n){
  km <- kmeans(def_scaled_cluster, centers = k, nstart = 20)

  wss[k] <- km$tot.withinss
}

wss_df <- tibble(clusters = 1:n, wss = wss)

elbow_plot <- ggplot(wss_df, aes(x = clusters, y = wss, group = 1))+
  geom_point(size = 2)+
  geom_line()+
  scale_x_continuous(breaks = seq(2, n, 2)) +
  xlab('Number of Clusters')+
  geom_hline(
    yintercept = wss,
    linetype = 'dashed'
  )

elbow_plot
```



As shown in each of the elbow plots above, there is no sharply defined 'elbow' that clearly indicates the optimal value of K, or the number of clusters to create for both offensive and defensive groupings. Fortunately,

given the exploratory and unsupervised nature of K-Means, the choice of K can be guided by both the visual evidence from the plots, as well as the goals of the analysis. Based on the trends observed in the plots, a reasonable range for K falls between approximately 5 and 14.

Based on the goals and interests of this projects, we chose to create K = 8 clusters for both offensive and defensive groupings. Several factors influenced this decision. First, since we were aiming to create a diverse set of team archetypes on both sides of the ball, we did not want to choose to small of a value of K, as this small number of clusters would not capture the unique differences between playing styles as well. In contrast, selecting too large a K would result in an effective overfitting of clusters, which would create difficulty in distinguishing and interpreting styles of the clusters, as well as reduce the number of teams per cluster to meaningfully analyze historical matchups.

With the value of K chosen as 8 for each cluster, we reran the K-Means algorithm with this value and a consistent random seed to create both offensive and defensive cluster assignments for each team.

Offense

```
k <- 8

set.seed(123)
km14 <- kmeans(off_scaled_cluster, centers = k, nstart = 20)


offensiveDataMin$FullCluster <- as.factor(km14$cluster)

off_scaled_df <- as.data.frame(off_scaled)
off_scaled_df$FullCluster <- as.factor(km14$cluster)
```

Defense

```
k <- 8

set.seed(123)
km14 <- kmeans(def_scaled_cluster, centers = 8, nstart = 20)


defensiveDataMin$FullCluster <- as.factor(km14$cluster)

def_scaled_df <- as.data.frame(def_scaled)
def_scaled_df$FullCluster <- as.factor(km14$cluster)
```

With offensive and defensive cluster assignments for each team established, our goal is to quantify how favorable or unfavorable specific matchups are between different offensive versus defensive archetype matchups. To achieve this, we developed a 'performance over expectation' (POE) metric for each offensive cluster defensive cluster pairing.

The idea is quite intuitive, and can be clearly illustrated through an example. Assume Team 1 belongs to offensive cluster 2 and defensive cluster 3, and Team 2 belongs to offensive cluster 1 and defensive cluster 7. In their matchup, Team 1 wins the game 85-70. According to KenPom's adjusted efficiency margin (AdjEM), which estimates how many points better or worse a team is than average, Team 1 has an AdjEM of 13 and Team 2 has an AdjEM of 3. Since the game is played on a neutral court, Team 1 is expected to win by 10 points (13 - 3). The actual margin of victory was 15 points, so Team 1 exceeded expectations by 5 points.

This 5 point performance over expectation contributes to both matchups within the game: Team 1's offense versus Team 2's defense, and Team 1's defense versus Team 2's offense. By performing such an analysis over

all games within the dataset, we can calculate the average performance over expectation for each unique offensive cluster versus defensive cluster pairing.

The result of this aggregation is a matrix, where each entry (i, j) represents the average points above (or below) expectation that teams in offensive cluster i score against teams in defensive cluster j. This matrix summarizes, in a numerical way, the matchup effectiveness and favorability between any offensive cluster defensive archetype combination.

Performance Over Expectation Analysis

```
netratings_19 <- read.csv("data/kpratings_19_pt.csv") %>% select(TeamName, Season, AdjEM)
netratings_20 <- read.csv("data/kpratings_20_pt.csv") %>% select(TeamName, Season, AdjEM)
netratings_21 <- read.csv("data/kpratings_21_pt.csv") %>% select(TeamName, Season, AdjEM)
netratings_22 <- read.csv("data/kpratings_22_pt.csv") %>% select(TeamName, Season, AdjEM)
netratings_23 <- read.csv("data/kpratings_23_pt.csv") %>% select(TeamName, Season, AdjEM)
netratings_24 <- read.csv("data/kpratings_24_pt.csv") %>% select(TeamName, Season, AdjEM)

netratings_all <- rbind(netratings_19, netratings_20, netratings_21,
                        netratings_22, netratings_23, netratings_24)

gamesData <- read.csv("data/gameResults.csv")
gamesData$date <- as.Date(gamesData$date)
gamesData$month <- as.numeric(format(gamesData$date, "%m"))
gamesData$year <- as.numeric(format(gamesData$date, "%Y"))
gamesData$season <- ifelse(gamesData$month %in% 1:4, gamesData$year - 1, gamesData$year)
gamesData <- na.omit(gamesData)
```

```
teamConversions <- c(
  "Arizona_St." = "Arizona_State",
  "California" = "Cal",
  "Florida_St." = "Florida_State",
  "Iowa_St." = "Iowa_State",
  "Kansas_St." = "Kansas_State",
  "Miami_FL" = "Miami",
  "Michigan_St." = "Michigan_State",
  "Mississippi" = "Ole_Miss",
  "Mississippi_St." = "Miss_State",
  "N.C._State" = "NC_State",
  "North_Carolina" = "UNC",
  "Ohio_St." = "Ohio_State",
  "Oklahoma_St." = "Oklahoma_State",
  "Oregon_St." = "Oregon_State",
  "Pittsburgh" = "Pitt",
  "Penn_St." = "Penn_State",
  "Washington_St." = "Washington_State",
  "Virginia" = "UVA"
)
```

```
getTeamRating <- function(teamName, season, netratings) {
  ratingRow <- netratings %>%
    filter(TeamName == teamName, Season == season)

  return(ratingRow$AdjEM)
}
```

```r
getMarginOverExpected <- function(teamName, opponentName, gameSeason, netratings, gamesData) {
  underscoreTeam <- gsub(" ", "_", teamName)
  underscoreTeam <- recode(underscoreTeam, !!!teamConversions)

  # Grab game(s) with these two teams
  gameRow <- gamesData %>%
    filter(team == underscoreTeam, opponent == opponentName, season == gameSeason)

  if(nrow(gameRow) == 0) {
    errStr <- paste("ERROR: Couldn't find game between", teamName, " and ", opponentName, " season ", ga
    #print(errStr)
    return(NA)
  }

  # Get KenPom Adj Efficiency Margin Ratings
  teamRtg <- getTeamRating(teamName, gameSeason, netratings)
  oppRtg <- getTeamRating(opponentName, gameSeason, netratings)

  margins <- c()
  for(i in 1:nrow(gameRow)){ # For each game

    # Get Team Score, Opponent Score, and Location
    teamScore <- gameRow$team_score[i]
    oppScore <- gameRow$opp_score[i]
    location <- gameRow$location[i]

    # Get Home Court Advantage Value
    homeCourtAdvantage <- 0
    if(location == "H") {
      homeCourtAdvantage <- 3
    } else if (location == "A") {
      homeCourtAdvantage <- -3
    }

    # Create Team Margin Over Expected
    actualMargin <- teamScore - oppScore

    #print(paste("           Team Rtg: ", teamRtg, " Opp Rtg: ", oppRtg))

    expectedMargin <- teamRtg - oppRtg + homeCourtAdvantage

    #print(paste("           Actual Margin: ", actualMargin, " Expected Margin: ", expectedMargin))

    marginOverExpected <- actualMargin - expectedMargin

    margins <- c(margins, marginOverExpected)
  }

  return(margins)
}
```

```r
numOffClusters <- length(unique(offensiveDataMin$FullCluster))
numDefClusters <- length(unique(defensiveDataMin$FullCluster))
margin <- matrix(0, nrow = numOffClusters, ncol = numDefClusters)
```

```r
num <- matrix(0, nrow = numOffClusters, ncol = numDefClusters)
perfOverExpect <- matrix(0, nrow = numOffClusters, ncol = numDefClusters)

for(i in 1:nrow(offensiveDataMin)) {
  teamName <- offensiveDataMin$TeamName[i]
  currSeason <- offensiveDataMin$Season[i]
  teamOffCluster <- offensiveDataMin$FullCluster[i]

  teamDefClusterRow <- defensiveDataMin %>%
    filter(TeamName == teamName, Season == currSeason)

  if (nrow(teamDefClusterRow) == 0) {
    warning(paste("Missing DEF cluster for team:", teamName, "season:", currSeason))
    next
  }

  teamDefCluster <- teamDefClusterRow$FullCluster[1]

  teamOpps <- gamesData %>%
    filter(team == teamName, season == currSeason)

  if (nrow(teamOpps) == 0) {
    #warning(paste("No games found for team:", teamName, "season:", currSeason))
    next
  }

  #print(paste("TEAM: ", teamName, " OFF: ", teamOffCluster, " DEF: ", teamDefCluster))

  for(j in 1:nrow(teamOpps)){
    oppName <- teamOpps$opponent[j]

    oppOffRow <- offensiveDataMin %>%
      filter(TeamName == oppName, Season == currSeason)

    oppDefRow <- defensiveDataMin %>%
      filter(TeamName == oppName, Season == currSeason)

    oppOffCluster <- oppOffRow$FullCluster[1]
    oppDefCluster <- oppDefRow$FullCluster[1]

    #print(paste("     OPP: ", oppName, " OFF: ", oppOffCluster, " DEF: ", oppDefCluster))

    if(is.na(oppOffCluster) | is.na(oppDefCluster)) {
      next
    }

    # Get Margin Over Expected values for games between these teams
    moes <- getMarginOverExpected(teamName, oppName, currSeason, netratings_all, gamesData)

    if (is.na(moes[1])) {
      next
    }
```

```
    for (moe in moes) {
      #print(paste("           MOE: ", moe))

      # Add only the Team Offensive Cluster versus Opp. Defensive Cluster Margin
      #   The other way around will be added when the opponent is traversed
      margin[teamOffCluster, oppDefCluster] <- margin[teamOffCluster, oppDefCluster] + moe
      num[teamOffCluster, oppDefCluster] <- num[teamOffCluster, oppDefCluster] + 1
    }
  }
}
```

```
# Create Performance Over Expectation "matrix"
for (k in 1:nrow(margin)) {
  for (m in 1:ncol(margin)) {
    if (num[k, m] > 0) {
      perfOverExpect[k, m] <- margin[k, m] / num[k, m]
    } else {
      perfOverExpect[k, m] <- NA  # Avoid division by zero
    }
  }
}
```

```
# Custom Cluster Names
offensiveClusterNames <- c(
  "Sloppy Foul-Prone Slashers",
  "Balanced Ball Controllers",
  "Average Perimeter Focused",
  "Elite Three Level Offense",
  "Inefficient Launchers",
  "Inefficient Paint Pounders",
  "Efficient Guard-Centric Perimeter",
  "Young Solid Rim Runners"
)

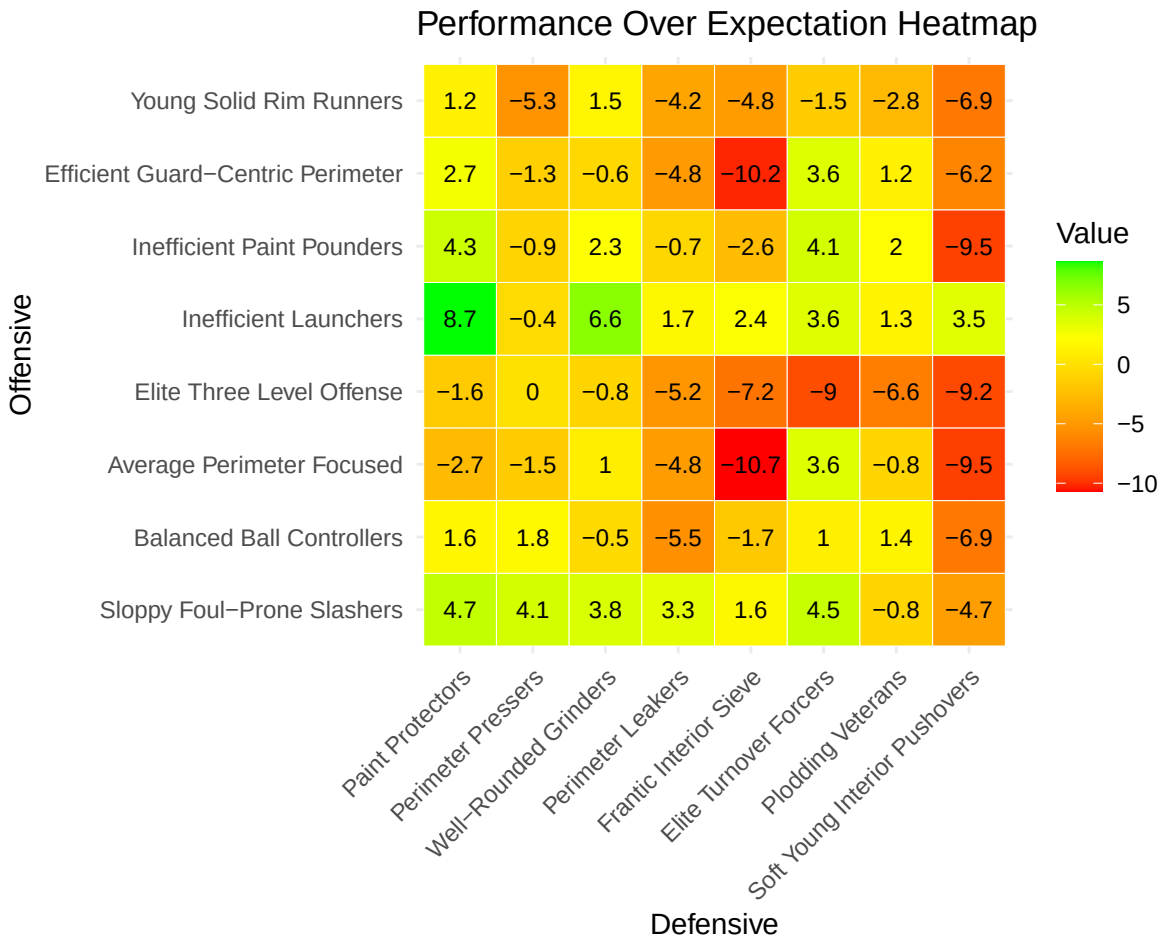
defensiveClusterNames <- c(
  "Paint Protectors",
  "Perimeter Pressers",
  "Well-Rounded Grinders",
  "Perimeter Leakers",
  "Frantic Interior Sieve",
  "Elite Turnover Forcers",
  "Plodding Veterans",
  "Soft Young Interior Pushovers"
)

rownames(perfOverExpect) <- offensiveClusterNames
colnames(perfOverExpect) <- defensiveClusterNames

perf_df <- melt(perfOverExpect)
colnames(perf_df) <- c("Offensive", "Defensive", "Value")

ggplot(perf_df, aes(x = Defensive, y = Offensive, fill = Value)) +
  geom_tile(color = "white") +
  scale_fill_gradientn(colors = c("red", "orange", "yellow", "green")) +
```

```
  geom_text(aes(label = round(Value, 1)), size = 3) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Performance Over Expectation Heatmap") +
  coord_fixed()
```



The performance over expectation matrix can be visualized above in the form of a heatmap, and for supplementary purposes, we have included a win percentage matrix with a similar approach.

With this analysis complete, we can now identify favorable and unfavorable matchups based on the numerically defined groupings of college basketball teams. The process begins by determining a team's offensive and defensive cluster assignments. Using these, we reerence the performance over expectation matrix (perfOverExpect) to assess the favorability, through performance over expectation historically, of the matchup.

To evaluate a team's offensive matchup favorabilities, we examine the row in the POE matrix corresponding to the team's offensive cluster. Positive values in this row indicate that teams within this offensive cluster outperform expectations against the defensive cluster represented by that columns index. Conversely, negative values indicate underperformance against that defensive archetype.

Evaluation of a team's defensive matchup favorability requires an inverted view. In this analysis, we examine the column in the POE matrix corresponding to the team's defensive cluster. We can see how various offensive archetype performs against this defensive cluster by evaluating entries of this column. In this case, negative values suggest that the team's defensive cluster tends to hold the respective archetype below expectation, indicating a defensive performance that is better than expectation.

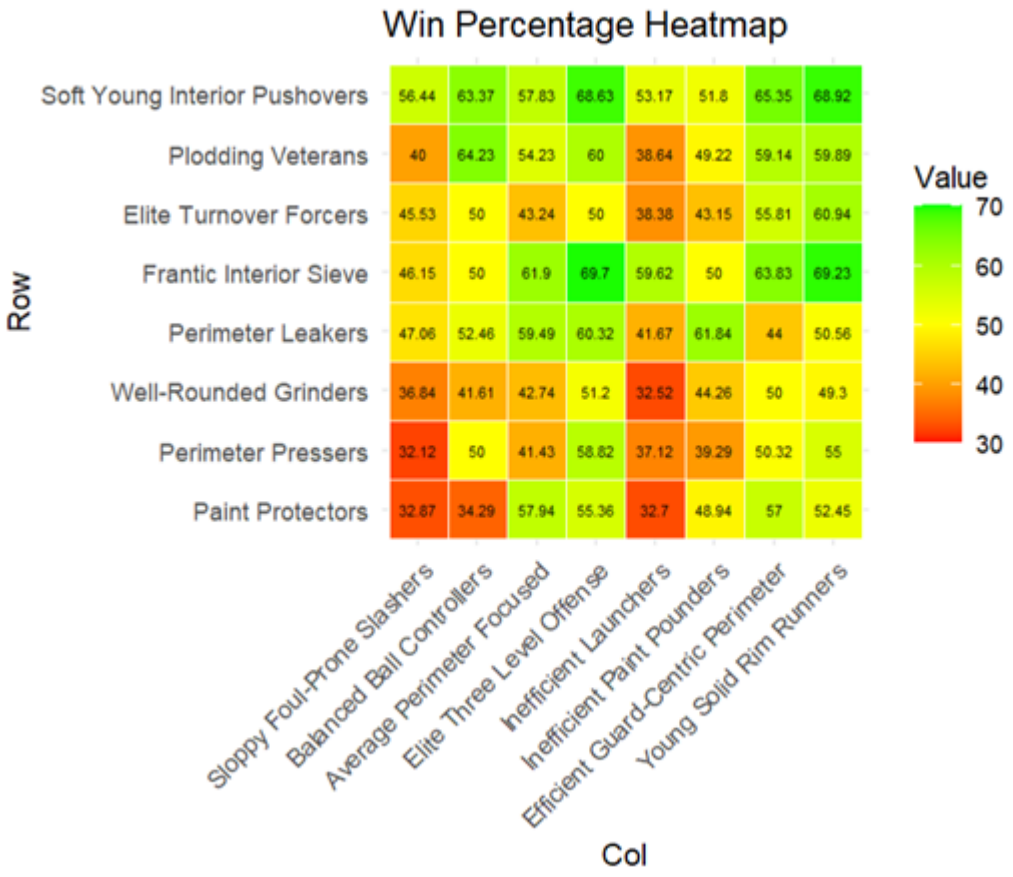


Figure 1: Win Percentage Matrix (in perspective of Offensive Cluster)

Given these contrasting, and somewhat easily confusable viewpoints of the POE matrix, a key focus was placed on developing clear visualizations to aid in understanding how different team archetypes perform in different matchups. Beyond this, we aimed to characterize the defining features of each cluster, allowing explanation of the shared traits or tendencies among teams of that cluster, and their strength within these defining characteristics. Below, we define a variety of supplementary functions that allow for visualization of such defining characteristics of clusters, with explanations succeeding each visualization.

Supplementary Functions

```
makeVarDf <- function(clusterN, scaled_df) {
  # Make data frame of variance reductions for clusterN

  # Step 1: Grab a single clusters data
  clusterNScaled <- scaled_df %>%
    filter(FullCluster == clusterN) %>%
    select(-FullCluster, -TeamName, -Season)

  # Step 2: Calculate Centered Variance for each feature
  cNVar <- 1 - sapply(clusterNScaled, var)

  # Step 3: Sort the variances
  cNVar_sorted <- sort(cNVar, decreasing = FALSE)

  # Step 4: Convert to data frame for plotting
  var_df <- data.frame(
    feature = factor(names(cNVar_sorted), levels = names(cNVar_sorted)),
    variance = as.numeric(cNVar_sorted)
  )

  return(var_df)
}
```

```
varReductionPlot = function(clusterN, scaled_df) {
  # Var. Reduction Analysis for identification of Important Cluster Features
  # If there is a large variance reduction of a feature from the variance throughout
  # all observations to the variance of the feature within a cluster, then we define
  # that statistic to be a 'substantial variance reducing feature'

  # Make data frame of variance reductions for clusterN
  var_df <- makeVarDf(clusterN, scaled_df)

  # Plot in Decreasing variance reduction
  defVarRed <- ggplot(var_df, aes(x = feature, y = variance)) +
    geom_bar(stat = "identity", fill = "skyblue") +
    theme_minimal() +
    ylab("Variance") +
    xlab("Feature") +
    ggtitle(paste("Feature Variance Decreases (Cluster", clusterN, ")", sep = " "))+
    scale_y_continuous(labels = scales::comma) +
    coord_flip() +
    theme(
      axis.text.y = element_text(
        size = 10,
        angle = 0,
```

```
        hjust = 0,
        vjust = 0.5,
        margin = margin(b = 15)
      ),
      axis.text.x = element_text(angle = 0, hjust = 1) # Ensure x-axis labels are readable
    )

  return(defVarRed)
}
```

```
makeCompTable <- function(clusterN, scaled_df){
  # Make table that compares stats within cluster to out of cluster
  ## NOTE: Change scaled_df to {offensive,defensive}Data for true values

  # Make data frame of variance reductions for clusterN
  var_df <- makeVarDf(clusterN, scaled_df)

  # Step 4: Convert to data frame for plotting
  var_df_sorted <- var_df %>%
    arrange(-variance) %>%
    filter(variance > 0.25)

  # Stats with significantly reduced variance
  var_reduced_stats <- var_df_sorted$feature

  # Every team's reduced variance stats, with their cluster
  var_reduced_values <- scaled_df %>%
    select(all_of(var_reduced_stats), FullCluster)

  # Current cluster's reduced variance stats (team by team)
  clusterNValues <- var_reduced_values %>%
    filter(FullCluster == clusterN)

  # Non-current cluster's reduced variance stats (team by team)
  nonClusterNValues <- var_reduced_values %>%
    filter(FullCluster != clusterN)

  # Average of stats within cluster
  clusterNAvgs <- clusterNValues %>%
    group_by(FullCluster) %>%
    summarise(
      across(everything(), mean, na.rm = TRUE)
    )

  # Average of stats outside of cluster
  nonClusterNAvgs <- nonClusterNValues %>%
    summarise(
      across(-FullCluster, mean, na.rm = TRUE)
    )

  variance_reductions <- as.data.frame(t(setNames(var_df_sorted$variance, var_df_sorted$feature)))

  clusterNTitle <- paste("Cluster", clusterN, sep = " ")
  nonClusterNTitle <- paste("Non Cluster", clusterN, sep = " ")
```

```r
  # Comparison table: in cluster vs. out of cluster values, variance reductions
  comparisonTable <- bind_rows(
    clusterNAvgs %>% select(-FullCluster) %>% mutate(Type = clusterNTitle),
    nonClusterNAvgs %>% mutate(Type = nonClusterNTitle),
    variance_reductions %>% mutate(Type = "Variance Reduction")
  ) %>%
  select(Type, everything())

  # Row that is difference of in cluster vs. out of cluster values
  difference_row <- comparisonTable %>%
    filter(Type == clusterNTitle) %>%
    select(-Type) %>%
    as.data.frame() %>%
    `-`(comparisonTable %>% filter(Type == nonClusterNTitle) %>% select(-Type) %>% as.data.frame())

  # Bind all rows together
  comparisonTable <- bind_rows(
    comparisonTable,
    difference_row %>% mutate(Type = "Difference")
  ) %>%
    mutate(across(where(is.numeric), ~ round(.x, 2)))

  return(comparisonTable)
}
```

```r
## Credit: https://towardsdatascience.com/how-to-make-a-spider-chart-in-r-using-ggplot2-85a4f1898cab/

ggspider <- function(p_data, legend_title = "Group", background_color = "gray97",
area_fill = TRUE, central_distance = 0.2, axis_name_offset = 0.2, grid.min = 0, grid.max = 1){

  circle_coords <- function(r, n_axis = ncol(p_data) - 1){
    fi <- seq(0, 2*pi, (1/n_axis)*2*pi) + pi/2
    x <- r*cos(fi)
    y <- r*sin(fi)

    tibble(x, y, r)
  }

  (step_1 <- map_df(seq(0, 1, 0.25) + central_distance, circle_coords) %>%
      ggplot(aes(x, y)) +
      geom_polygon(data = circle_coords(1 + central_distance), alpha = 1, fill = background_color) +
      geom_path(aes(group = r), lty = 2, alpha = 0.5) +
      theme_void())

  axis_coords <- function(n_axis){
    fi <- seq(0, (1 - 1/n_axis)*2*pi, (1/n_axis)*2*pi) + pi/2
    x1 <- central_distance*cos(fi)
    y1 <- central_distance*sin(fi)
    x2 <- (1 + central_distance)*cos(fi)
    y2 <- (1 + central_distance)*sin(fi)

    tibble(x = c(x1, x2), y = c(y1, y2), id = rep(1:n_axis, 2))
  }
```

```r
text_data <- p_data %>%
  select(-group) %>%
  map_df(~ grid.min + (grid.max - grid.min) * seq(0, 1, 0.25)) %>%
  mutate(r = seq(0, 1, 0.25)) %>%
  pivot_longer(-r, names_to = "parameter", values_to = "value")

text_coords <- function(r, n_axis = ncol(p_data) - 1){
  fi <- seq(0, (1 - 1/n_axis)*2*pi, (1/n_axis)*2*pi) + pi/2 + 0.01*2*pi/r
  x <- r*cos(fi)
  y <- r*sin(fi)

  tibble(x, y, r = r - central_distance)
}

labels_data <- map_df(seq(0, 1, 0.25) + central_distance, text_coords) %>%
  bind_cols(text_data %>% select(-r))

labels_data_top_axis <- labels_data %>%
filter(parameter == names(p_data)[2])

rescaled_coords <- function(r, n_axis){
  fi <- seq(0, 2*pi, (1/n_axis)*2*pi) + pi/2
  tibble(r, fi) %>% mutate(x = r*cos(fi), y = r*sin(fi)) %>% select(-fi)
}

rescaled_data <- p_data %>%
  mutate(across(-group, ~ (.x - grid.min) / (grid.max - grid.min))) %>%
  mutate(copy = pull(., 2)) %>%
  pivot_longer(-group, names_to = "parameter", values_to = "value") %>%
  group_by(group) %>%
  mutate(coords = rescaled_coords(value + central_distance, ncol(p_data) - 1)) %>%
  unnest()

step_1 +
  geom_line(data = axis_coords(ncol(p_data) - 1),
            aes(x, y, group = id), alpha = 0.3) +
  geom_point(data = rescaled_data,
            aes(x, y, group = group, col = group), size = 3) +
  geom_path(data = rescaled_data,
            aes(x, y, group = group, col = group), size = 1) +
  {if(area_fill == TRUE) geom_polygon(data = rescaled_data,
                          aes(x, y, group = group,
                              col = group, fill = group),
                              size = 1, alpha = 0.05, show.legend = FALSE)} +
  geom_text(data = labels_data_top_axis,
                   aes(x, y, label = value), alpha = 1, size = 5) +
  geom_text(data = text_coords(1 + central_distance + axis_name_offset),
            aes(x, y), label = labels_data$parameter[1:(ncol(p_data)-1)]) +
  labs(col = legend_title) +
  theme(legend.position = "bottom",
        legend.text = element_text(size = 12),
        legend.title = element_text(size = 12)) +
  coord_fixed()
```

```
}
```

```
makePolyPlot <- function(clusterN, scaled_df) {
  radarData <- makeCompTable(clusterN, scaled_df) %>%
  filter(Type %in% c(paste("Cluster", clusterN), paste("Non Cluster", clusterN))) %>%
  rename(group = Type)

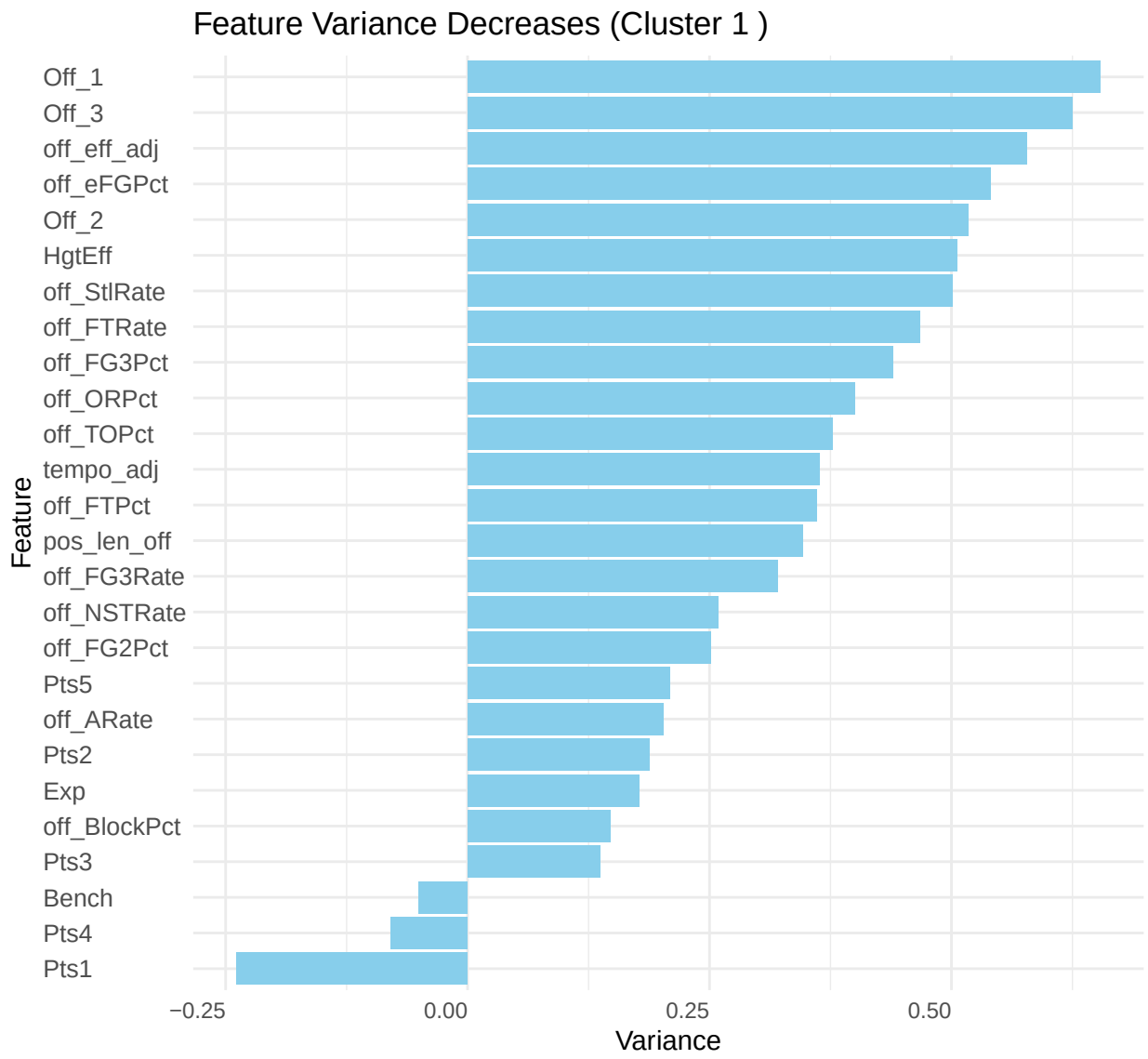
  spiderPlot <- ggspider(radarData, grid.min = -2, grid.max = 2)

  return(spiderPlot)
}
```

Analysis of Charactaristics for an Example Team

Let's assume a team is of offensive cluster 1. Below, we can see two plots that assist in understanding of each of these cluster's defining characteristics.

```
varReductionPlot(1, off_scaled_df)
```



The plot above illustrates the 'variance reductions' for normalized statistic values, comparing teams in offensive cluster 1 to the full population of teams. Since each statistic is normalized, each has a variance of 1 across all teams. To compute the variance reduction for a given metric within a specific cluster, we simply calculate: Variance Reduction = 1 - variance of that statistic for teams within the cluster of interest.

The purpose of this metric is to help to identify which statistics are 'defining characteristics' of the cluster. The underlying intuition is that if teams within a cluster show lower variance for a particular statistic than the whole dataset does, that statistic most likely represents a shared trait among teams in that cluster. Importantly, this does not indicate whether the cluster performs better or worse in that statistic than average, it only means that teams within that cluster perform similarly for that metric.

As seen above, several statistics emerge as 'defining characteristics' for offensive cluster 1. Particularly, Off_1 and Off_3 (proportion of points from free throws and three pointers, respectively), off_eff_adj (adjusted offensive efficiency), and off_eFGPct (effective field goal percentage), among others, show meaningful variance reduction, and are all statistics which teams in offensive cluster 1 have similar values in.

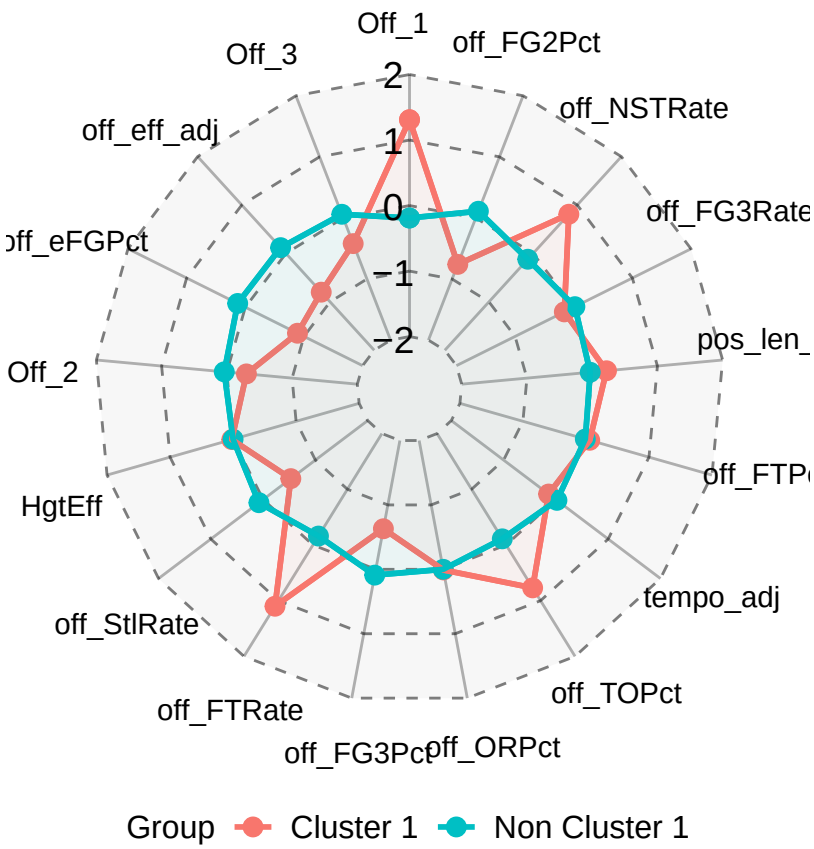
To more fully understand characteristics of a cluster and their 'defining characteristics', we can generate a polygon plot that shows the average standardized values of statistics with a variance reduction of 0.25 or greater. This visual allows comparison of the how the cluster at hand performs in key metric areas as compared to the rest of the dataset.

```
makePolyPlot(1, off_scaled_df)
```

```
## Warning: There was 1 warning in `summarise()`.
## i In argument: `across(everything(), mean, na.rm = TRUE)`.
## i In group 1: `FullCluster = 1`.
## Caused by warning:
## ! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
## Supply arguments directly to `.fns` through an anonymous function instead.
##
##   # Previously
##   across(a:b, mean, na.rm = TRUE)
##
##   # Now
##   across(a:b, \(x) mean(x, na.rm = TRUE))
```

```
## Warning: `cols` is now required when using `unnest()`.
## i Please use `cols = c(coords)`.
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



This polygon plot displays the average standardized values of each defining characteristic for offensive cluster 1 (shown in red) compared to the overall dataset average for each metric (shown in blue). With this visual comparison, we can better understand the strengths and weaknesses of this cluster.

Notably, offensive cluster 1 exhibits a significantly higher Off_1 value, indicating a greater proprtion of their points come from free throws compared to an average team. This is further supported by an elevated off_FTRate (free throw rate). Additionally, this cluster shows high values in off_TOPct (turnover percentage) and off_NSTRate (non-steal turnover rate), suggesting a tendency towards sloppy offensive play and high frequency of turnovers. On top of these issues, we see that the cluster has a very low off_eff_adj (adjusted offensive efficiency) compared to the average team, indicating that teams within this cluster are generally poor offenses.

Based on these patterns, we might label this archetype "Sloppy Slashers", reflecting their reliance on getting to the free throw line to score, but also their poor ball controll and overall offensive inefficiency.

A similar analysis can be performed across all offensive and defensive clusters. This two-faceted analysis approach not only allows us to quantify matchup effectiveness using performance over expectation metrics, but also enhances our understanding of the statistical traits that define each cluster, connecting numerical similarities to real-life team identities.

Team and Cluster Visualization Dashboard

In addition to the above visualizations, we have created an interactive Shiny Dashboard, which allows a more clear and holistic visualization for matchup analysis and identification of 'defining characteristics' for the various offensive and defensive clusters. This application is located within the folder CBB_Archetype_Dashboard, and can be ran simply by opening the app.R file and executing the script.

Results

The process of clustering college basketball teams into offensive and defensive clusters has provided us with a multitude of meaningful deliverable results. First and foremost, our archetype matchup 'matrix' acts as a basis for understanding how any defined offensive 'archetype' performs (over expectation) against each defensive 'archetype'. Additionally, the emphasis on identification of defining characteristics of each cluster allows for a deeper understanding of how teams in college basketball are differentiated. Finally, our visualization dashboard acts as an interface for fans, analysts, and team faculty alike to use to understand the breadth of our analysis.

A multitude of impacts throughout college basketball analysis, team building, and strategy come as a result of these deliverables. First, from the perspective of analyzing team performance, a fan or analyst is able to make statistically supported claims about favorable or unfavorable matchups for a specific team. From both the perspective of a team's offensive and defensive archetype, historical performance results tell us what sorts of matchups prove to challenge (or benefit) specific teams based on their play style. Furthermore, a deeper dive into the defining characteristics of an archetype of interest can provide insight into what makes this offense or defense what it is, understanding its flaws and strengths, analyzing what differentiates it from other teams, and understanding why good and bad matchups exist for a team.

Additionally, from the perspective of uses for a college basketball team, the value of understanding strengths and weaknesses on both sides of the ball, as well as matchups in which your team might struggle, is clear. These insights may provide additional uses in team building and recruiting, development focuses, and play style changes, in order to optimize team performance. On top of this, there is clear value in understanding these same defining characteristics and matchup favorabilities when scouting and preparing for opposing teams. In analyzing what types of teams one's opponent may particularly struggle against, a team may change their game plan to take advantage of these struggles seeking to give their team the best chance.

Conclusion

After performing clustering of college basketball teams into offensive and defensive 'archetypes', analyzing defining characteristics within these archetypes, and understanding how each archetype performs relative to expectations against opponents, we are left with a lot of valuable insight. First, we come to realize that after purely grouping teams numerically using advanced metrics, we receive groupings that make sense in the context of basketball. With factors including overall efficiency, pace of play, scoring distribution, experience, shot usage, and more, these defined 'archetypes' are similar to the perceived groupings of team styles that exist within college basketball. By analyzing the importance and consistency of various metrics within each cluster, we can translate the results of the groupings into cluster names and descriptions that align with public perspectives of college basketball 'team builds'. In viewing performance of a cluster versus each opposing archetype, we come to the conclusion that for each archetype, there are types of teams that they perform both over and under expectation against, supporting the notion that favorable and unfavorable matchups do exist for every team depending on their archetype. In totality, the grouping of college basketball teams into offensive and defensive archetypes, understanding each archetype's defining characteristics, strengths, and weaknesses, and understanding how each archetype performs against others, we have produced a new framework that supports the analysis of how advanced metrics provide insight to game results in college basketball.

Future Work

Despite a thorough analysis of how college basketball teams can be grouped into offensive and defensive clusters, and how these clusters influence matchup results, a variety of improvements and expansions can be made to create additional valuable insight that supplements our analysis.

First, we could expand upon this project by diving deeper into the modeling aspect that backs the clustering process. Specifically, K-means clustering, the technique used in this study, is a widely used technique when it comes to unsupervised clustering models. K-means clustering allows for team grouping based on advanced analytics as desired, but it is not the only approach that exists. Going through a more thorough model

selection process would allow us to more solidly decide if the number of clusters chosen was correct and potentially leave a lot less up to interpretation. Though our goal was to create team groupings that made sense in a basketball context, ensuring that we are most efficiently splitting teams into clusters based on the features used is of utmost importance. Alongside a more methodical model selection process, variable selection and exploratory analysis on our advanced metrics used as variables could supplement our clustering process. Specifically, more advanced methods, including feature transformations and a penalized clustering algorithm could allow for more distinctive cluster creation, as well as tuning in on the most important team metrics that allow groupings of college basketball teams.

Additionally, we could make a variety of advancements on the framework already obtained within this study to advance the insights and usability in practice. Three such ideas come to the forefront among many possibilities: cluster similarity quantification, cluster matchup win simulation, and team archetype persistence.

The concept behind cluster similarity quantification is that based on the methodology behind K-Means clustering, though a specific observation (team) can be marked as closest to being one specific cluster, it may not be 'far' from being another cluster, or can be geometrically 'between' defined clusters. Because of this geometrical definition of clusters, we are able to define cluster similarity for each team based on how far away they are from each cluster center. This analysis allows us to not bucket a team into one specific 'archetype', but rather understand where teams may be a blend of two plus archetypes. This analysis allows us to deepen our understanding of how a team may match up with opponents of varying play styles from the perspective of this 'blended' archetype.

Next, the ability to "simulate matchups between clusters" may be useful to determine true expected results of matchups between two archetypes. Though this is more advanced, concepts including bootstrapping and Monte Carlo simulation would allow this process, and supplement the understanding of interactions between offensive and defensive archetypes.

Finally, and maybe most interestingly, is the concept of quantifying team archetype persistence. Specifically, the idea is that in our study, schools across each season in the dataset are treated as separate entities. In reality, teams and programs should have some sort of structure and persistence in team build, play style, strengths and weaknesses, etc. This may differ, though, depending on the team, as some teams recruit towards a play style, and others play based on their team's strengths. Using cluster similarity for schools across years, we could provide quantitative analysis on how much teams change in archetype from year to year, leading possibly to an advanced understanding of recruiting strategies, program philosophies, and more.